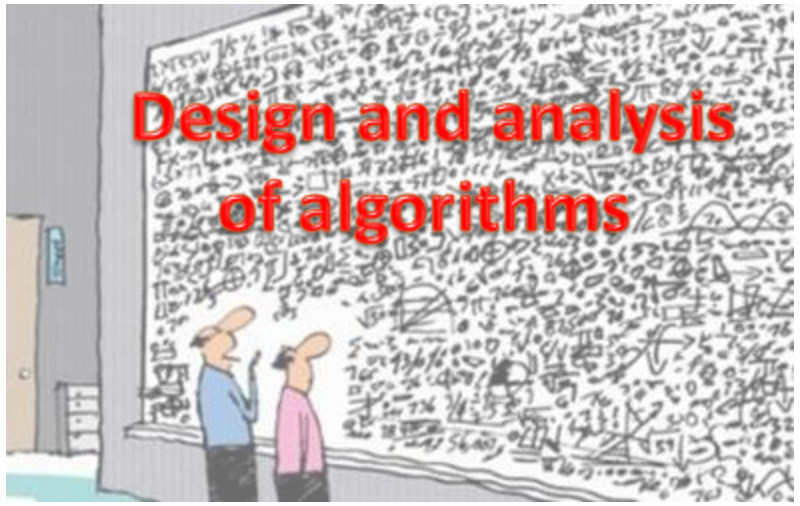


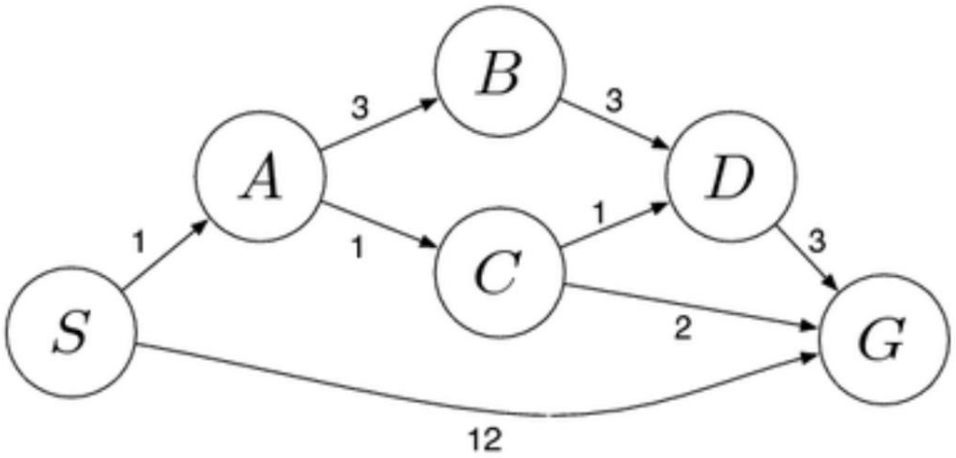
- **Uniform Cost Search:**
  A goal-oriented BFS

Objective

- **Extend BFS** to find shortest paths from origin s to a **given destination d** for arbitrary link weights. In classical BFS edges have unit distance ("hops" count).
- New interpretation: build a set of paths starting from s instead of a set of nodes.
- At the end of step $k-1$: P_{k-1} = current "potentially interesting" set of paths (if d is reachable, then **one of these path** is the **prefix of the shortest path** we are searching for).
- In step k we add more paths by expanding the most promising among the current paths: we choose the shortest path π^*_{k-1} from P_{k-1}, remove it out from the current set, and add the new paths that result by adding the neighbours of its leaf (final node). If we reach d, we record the path length w^*. We found a feasible path, but perhaps we can do better!
- We keep expanding $P_\ell, \ell > k$ by always expanding its shortest path while improving our shortest path length w^* when possible. We terminate if all candidate paths are $\geq w^*$.
- Classic BFS: Think of the queue as containing the leaves of the potentially interesting paths. Key difference: the first time we reach d in classic BFS we can stop (no way to beat that)

Example

- We want to find least cost path from **start** state S to **goal** state G
- Costs are given on the edges
- Cannot find best path by inspection for graphs with 100 nodes…
- We will use Uniform Cost Search (UCS)



Terminology

- H: list of currently explored paths with their total cost
- E: set of nodes that have already be expanded (i.e., we have considered the paths through them to their neighbours) and hence do not have to be considered any more

- H=data structure for storing paths with costs
- E= expanded node set
- Init: H={ (S:0) }, E={}
- Step1:
  - Take least cost path off H: H={}, p=S
  - expand leaf S of path=S: A,G
  - Add leaf to expanded node set E: E={S}
  - Throw away neighbors if in E = ok ☺
  - Check paths for cycles: ok ☺
  - compute path costs:
  - S-A:1, S-G:12
  - Add new paths:
    - H={S-G:12, S-A:1}, E={S}

we won't consider this path anymore since we will expand this further. p is this path

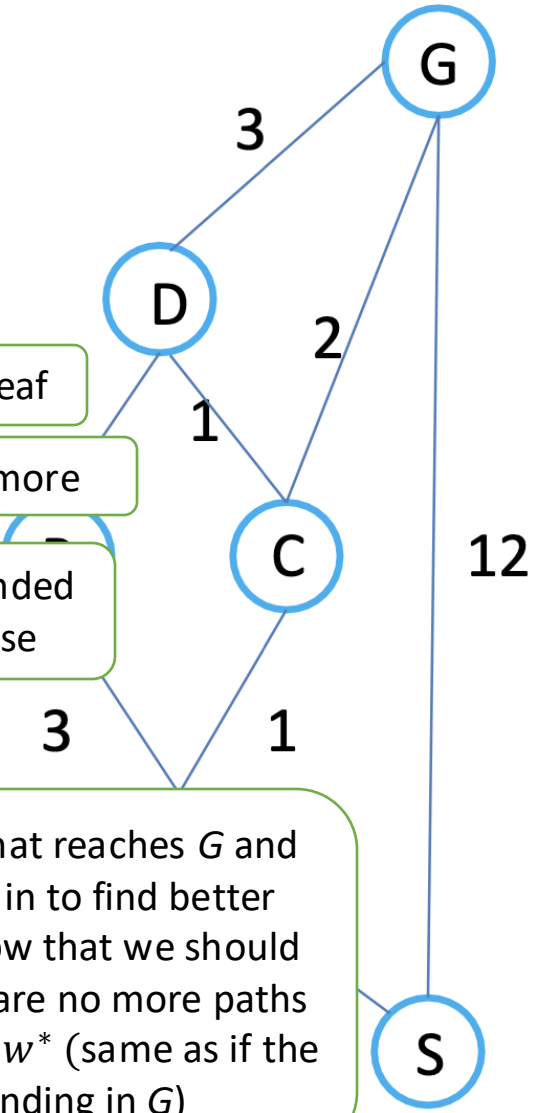
path: sequence of nodes

leaf: last node

these are the neighbors of the leaf

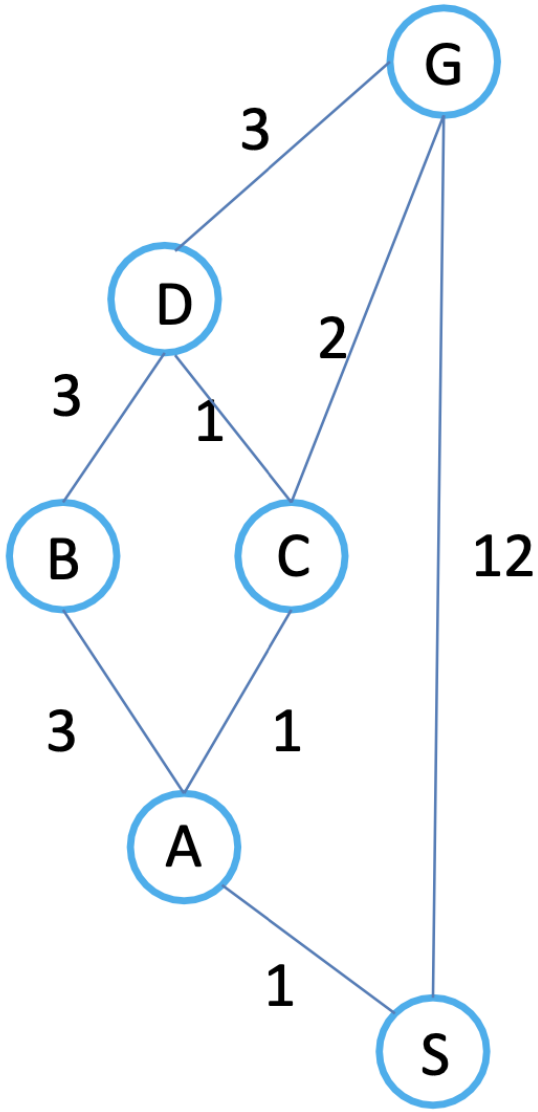
don't need anymore

what ever is expanded is of no future use



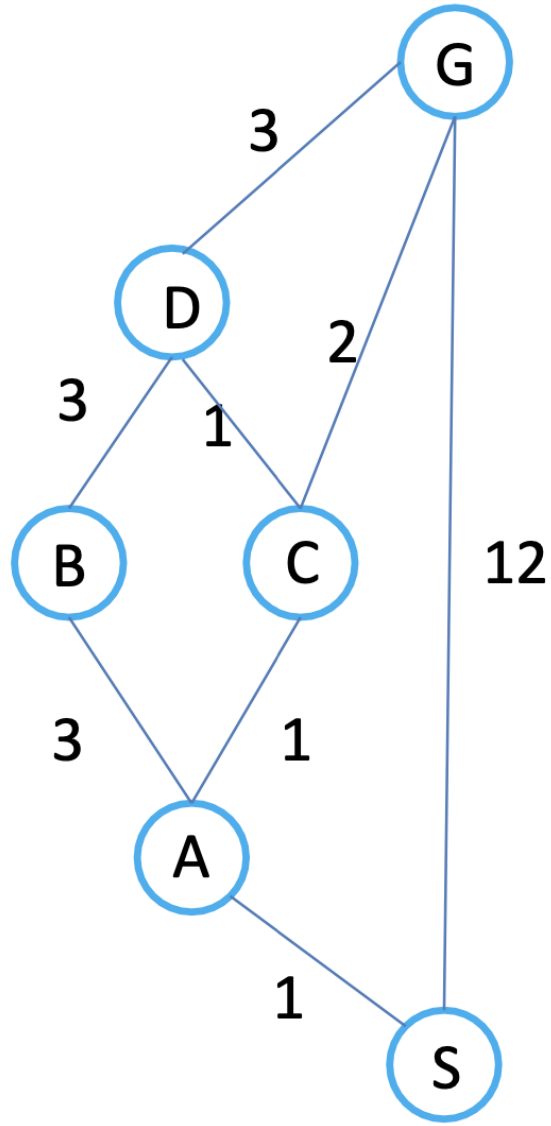
Note that we already found a path that reaches *G* and hence $w^* = 12$. But we keep going in to find better ones and reduce w^*. How do we know that we should not look any further? Either if there are no more paths to expand, or the paths in H are all $\geq w^*$ (same as if the current shortest path in H is ending in *G*)

- H=data structure for storing paths with costs
- E= expanded node set
- Init: H={ (S:0) }, E={}
- Step1:
  - Take least cost path off H: H={}, p=S
  - expand leaf S of path=S: A,G
  - Add leaf to expanded node set E: E={S}
  - Throw away neighbors if in E = ok ☺
  - Check paths for cycles: ok ☺
  - compute path costs:
  - S-A:1, S-G:12
  - Add new paths:
    - H={S-G:12, S-A:1}, E={S}



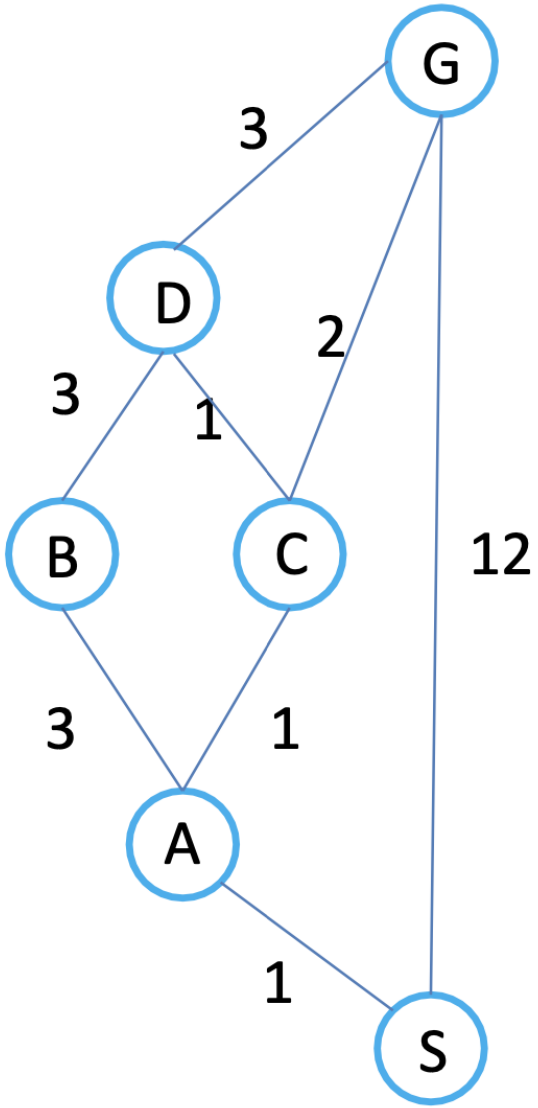
- H=data structure for storing paths with costs
- E= expanded node set

- Step2: H={S-G:12, S-A:1}, E={S}
  - Take least cost path off H: H={S-G:12}, p=S-A
  - expand leaf of path=S-A: S, B, C
  - Add leaf to expanded node set E: E={S,A}
  - Throw away neighbors if in E : S out!
  - Check paths for cycles: ok ☺
  - compute path costs:
  - S-A-B:4, S-A-C:2
  - Add new paths:
    - H={S-G:12, S-A-B:4, S-A-C:2}, E={S,A}



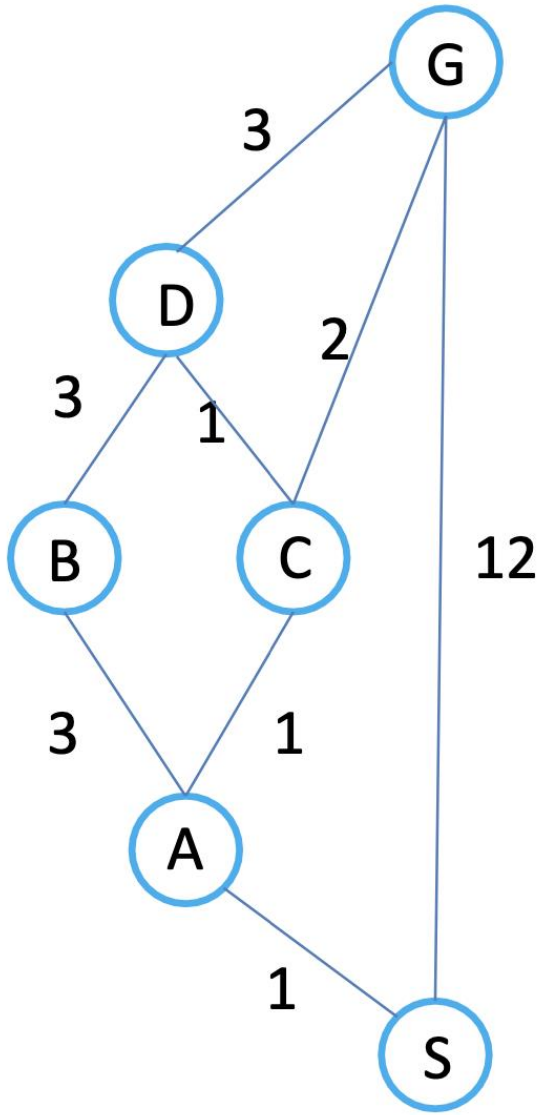
- H=data structure for storing paths with costs
- E= expanded node set

- Step3: H={S-G:12, S-A-B:4, S-A-C:2}, E={S,A}
  - Take least cost path off H: H={S-G:12, S-A-B:4}, p=S-A-C
  - expand leaf of path=S-A-C: A, D, G
  - Add leaf to expanded node set E: E={S,A,C}
  - Throw away neighbors if in E : A out!
  - Check paths for cycles: ok ☺
  - compute path costs:
  - S-A-C-D:3, S-A-C-G:4
  - Add new paths:
    - H={S-G:12, S-A-B:4, S-A-C-G:4, S-A-C-D:3}, E={S,A,C}



- H=data structure for storing paths with costs
- E= expanded node set

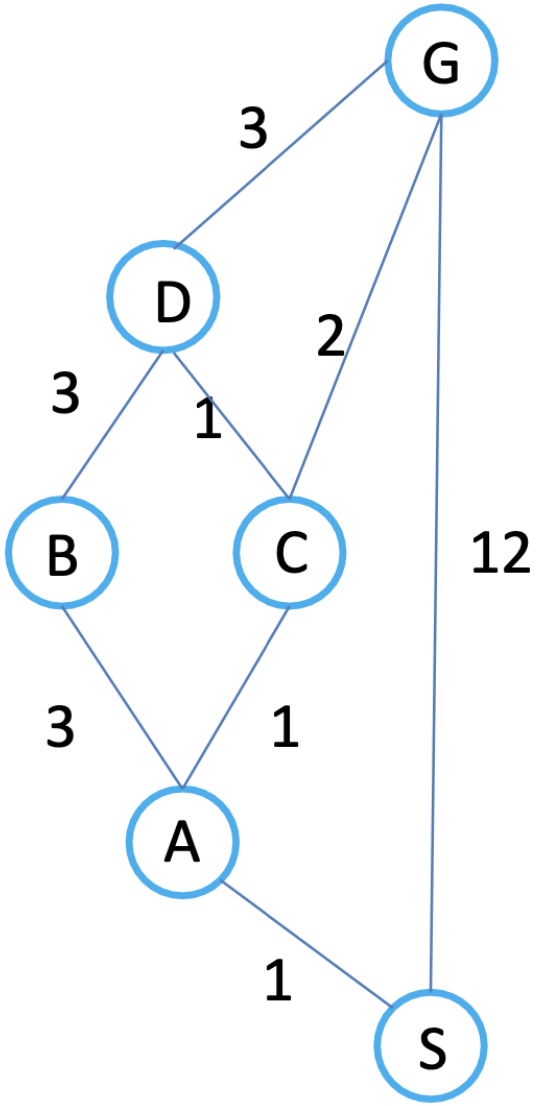
- Step4: H={S-G:12, S-A-B:4, S-A-C-G:4, S-A-C-D:3}, E={S,A,C}
  - Take least cost path off H: p=S-A-C-D
  - expand leaf of path=S-A-C-D: B, C, G
  - Add leaf to expanded node set E: E={S,A,C,D}
  - Throw away neighbors if in E : C out!
  - Check paths for cycles: ok ☺
  - compute path costs:
  - S-A-C-D-B:6, S-A-C-D-G:6
  - Add new paths:
    - H={S-G:12, S-A-C-D-B:6, S-A-C-D-G:6, S-A-B:4, S-A-C-G:4 }, E={S,A,C,D}



- H=data structure for storing paths with costs
- E= expanded node set

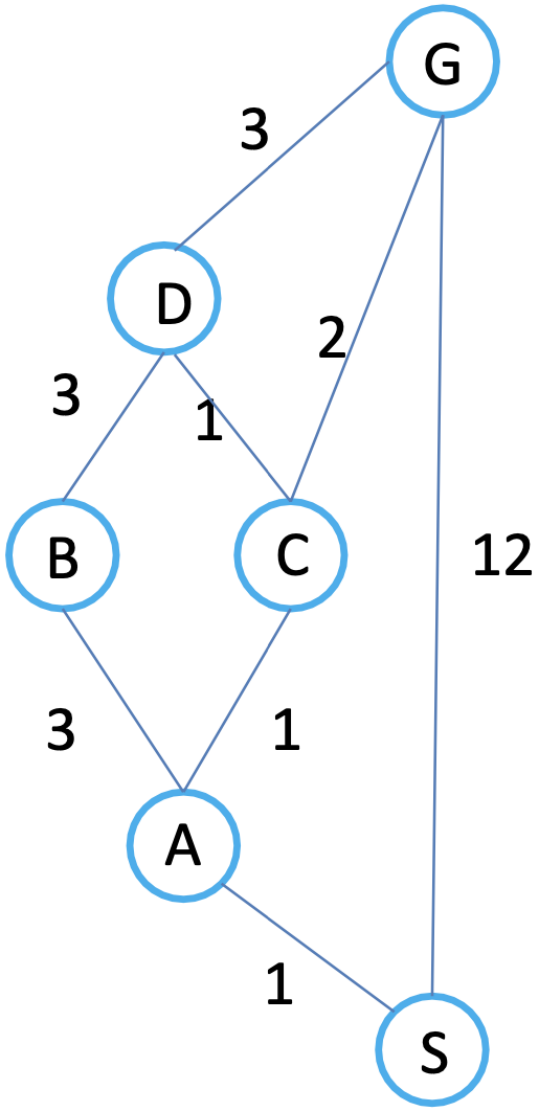
- Step5: H={S-G:12, S-A-C-D-B:6, S-A-C-D-G:6, S-A-B:4, S-A-C-G:4 }, E={S,A,C,D}
- Have a least cost tie, assume we take S-A-B off:
  - Take least cost path off H: p=S-A-B
  - expand leaf of path=S-A-B: A, ~~C~~, D
  - Add leaf to expanded node set E: E={S,A,C,D,B}
  - Throw away neighbors if in E : A,~~C~~,D out!
  - Add new paths:
    - Nothing to be done
    - H={S-G:12, S-A-C-D-B:6, S-A-C-D-G:6, S-A-C-G:4 }, E={S,A,C,D,B}

If we would take SACG off: cannot expand further



- H=data structure for storing paths with costs
- E= expanded node set

- Step6: H={S-G:12, S-A-C-D-B:6, S-A-C-D-G:6, S-A-C-G:4 }, E={S,A,C,D,B}
  - Take least cost path off H: p=S-A-C-G
  - Goal test: leaf node is goal. Algorithm returns: S-A-C-G:4



- **Complexity:** $O(m^{1+\lfloor \frac{l}{e} \rfloor})$ where:
  m is the maximum number of neighbors a node has
- l is the length of the shortest path to the goal state
- e is the least cost of an edge

```
UCS(G,start node s)
1:  H=min-heap(), Expanded={}
2:  H.insert(   (path(s):0)    )

3:  while true == true :
4:      if H.empty():
5:          return FAIL
6:      somepath=H.extract_min()  #removes somepath from the heap H!
7:      if goal-test(somepath) == true:
8:          return somepath          #found!
9:      leafnode=somepath.leaf()
10:     Expanded.insert ( leafnode  )
11:     for each n in G.Adj( leafnode ):
12:         if n in Expanded :
13:             pass
14:         if n NOT in Expanded :
15:             newpath=somepath * n   # * creates a new path
16:             newpath.cost= somepath.cost + edgecost(leafnode,n)
17:             H.insert( (newpath: newpath.cost) )
```

BFS for node discovery	UCS for path to goal state
Define a data structure "S" for storing nodes • Initialize with start node S	Define a data structure "S" for storing paths • Initialize with path= {S}
Pop() a vertex a from the data structure S with lowest distance, (pop = it is deleted from it!)	Pop() a path p from the data structure S with lowest total cost, (pop = it is deleted from it!) • add leaf node of the path to the expanded set
Terminate if: S is empty when Pop() is called	• Terminate if: Pop() returns a path p such that its leaf is a goal state
Get neighbor vertices of vertex a • check if neighbor vertices are visited already, • throw away neighbor vertices if visited	Get neighbor vertices of leaf node of path p, • Check if neighbor vertices are in the "expanded nodes" set, • throw away neighbor vertices if expanded
for each remaining neighbor vertex v • compute distance from start d(v)=d(a)+1 • Add vertex v to the visited set • Insert neighbor vertex with d(v) into data structure S	for each remaining neighbor vertex v • Construct a new path: old path p with leaf and the neighbor v appended, • Compute path cost for the new path • Insert new path into data structure

References

- https://inst.eecs.berkeley.edu/~cs188/sp20/assets/lecture/lec-3.pdf